

SIMATS ENGINEERING

CO5-ASSESSMENT TOOL1-Design Task

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	192312443
Name	P.Ramesh

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 40%

Time Duration: 1 Hour

QUESTIONS: 5

Total Marks:50

Code of Conduct:

I P. Ramesh (192312443) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P. Ramesh

1. End-to-End System Design

Design a scalable AI-based system using **PySpark and RDD operations** to analyze real-time social media data and generate intelligent recommendations, including system architecture, data flow, and agent integration.

2. Distributed Intelligent Agent System

Develop a **multi-agent AI system architecture** that uses PySpark for distributed data processing and enables agents to collaborate in solving a large-scale optimization problem.

3. Big Data AI Pipeline Design

Design a complete **distributed AI pipeline** using RDD transformations for data preprocessing, feature engineering, and model training, ensuring scalability and fault tolerance.

4. Real-Time Decision-Making System

Construct an AI-based system integrating **RDD-based streaming data processing** with intelligent agents for real-time decision-making in applications such as smart traffic or healthcare.

5. Cloud-Based Distributed AI Application

Design a **cloud-enabled AI system** using PySpark and intelligent agent architectures that supports large-scale data analytics, dynamic resource allocation, and adaptive learning.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined

Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation

1)ANS:

Introduction

The objective is to design an AI-based distributed system using PySpark, RDD operations, and intelligent agents to process real-time social media data and generate personalized recommendations.

The system collects posts, comments, likes, shares, hashtags, and user interactions. PySpark processes this large-scale data in parallel, while an AI recommendation agent analyzes user interests and selects relevant content.

System Objectives

The proposed system aims to:

1. Collect real-time social media data.
2. Process large-scale data using PySpark RDDs.
3. Extract useful features from social media content.

Architecture

Social Media Data



Real-Time Data Collection



PySpark + RDD



Data Cleaning & Feature Extraction



AI Recommendation Model



Intelligent Agent



Personalized Recommendations



User Feedback

RDD Operations

- `map()` – transforms data
- `filter()` – removes unwanted data
- `flatMap()` – extracts words/features
- `reduceByKey()` – aggregates data

Agent Integration

The intelligent agent:

1. Observes user activity.
2. Identifies user interests.
3. Ranks suitable content.
4. Provides recommendations.
5. Updates recommendations using user feedback.

Advantages

- Handles large-scale data.
- Supports distributed processing.
- Provides personalized recommendations.
- Scalable and fault-tolerant.
- Supports real-time decision-making.

Conclusion: PySpark processes social-media data efficiently, RDDs enable distributed computation, and the intelligent agent provides adaptive personalized recommendations.

2)ANS:

Objective

Develop a multi-agent AI system architecture using PySpark for distributed data processing, where multiple intelligent agents collaborate to solve a large-scale optimization problem.

Architecture

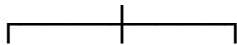
Large Dataset



PySpark



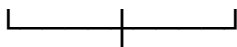
RDD Partitioning



Agent 1 Agent 2 Agent 3



Local Optimization



Agent Collaboration



Coordinator Agent



Global Optimal Solution

Main Components

PySpark Data Processing

- Loads large datasets using Spark.
- Cleans and preprocesses the data.
- Divides data into distributed partitions.
- Performs parallel computations using Spark.

Agent 1 – Planning Agent

- Analyzes the optimization problem.
- Creates possible solutions.
- Sends candidate solutions to other agents.

Agent 2 – Resource Agent

- Analyzes available resources.
- Determines resource constraints.
- Suggests efficient resource allocation.

Agent 3 – Optimization Agent

- Evaluates candidate solutions.
- Calculates objective/cost functions.
- Searches for improved solutions.

Agent Coordinator

- Collects results from all agents.
- Exchanges information between agents.
- Selects the best candidate solution.
- Maintains global coordination.

PROGRAM:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col
# Create Spark session
spark = SparkSession.builder \
    .appName("DistributedIntelligentAgentSystem") \
    .master("local[*]") \
    .getOrCreate()
# Sample large-scale optimization data
data = [
    (1, 100, 10),
    (2, 200, 20),
    (3, 150, 15),
```

```
(4, 300, 30),  
(5, 250, 25),  
(6, 400, 40),  
(7, 350, 35),  
(8, 500, 50)
```

```
]
```

```
df = spark.createDataFrame(  
    data,  
    ["item", "resource", "cost"]  
)
```

```
print("Input Data:")  
df.show()
```

```
# Partition data for distributed processing  
rdd = df.rdd.repartition(3)
```

```
# Agent 1: Planning Agent
```

```
def planning_agent(row):  
    return (  
        row.item,  
        row.resource,  
        row.cost,  
        row.resource / row.cost  
    )
```

```
planning_result = rdd.map(planning_agent)
```

```
# Agent 2: Resource Agent
```

```
def resource_agent(row):  
    if row[1] <= 250:  
        status = "Available"  
    else:  
        status = "High Resource"  
    return (row[0], row[1], row[2], status)
```

```
resource_result = rdd.map(resource_agent)
```

```

# Agent 3: Optimization Agent
def optimization_agent(row):
    score = row[1] / row[2]
    return (row[0], row[1], row[2], score)
optimization_result = rdd.map(optimization_agent)

# Collect optimization results
results = optimization_result.collect()

print("\nAgent 3 - Optimization Results:")
for result in results:
    print(result)
# Select best solution
best = max(results, key=lambda x: x[3])
print("\nBest Solution:")
print("Item:", best[0])
print("Resource:", best[1])
print("Cost:", best[2])
print("Optimization Score:", best[3])
spark.stop()

```

OUTPUT:

```

Input Data:
+-----+-----+-----+
|item|resource|cost|
+-----+-----+-----+
| 1 |    100 |  10 |
| 2 |    200 |  20 |
| 3 |    150 |  15 |
| 4 |    300 |  30 |
| 5 |    250 |  25 |
| 6 |    400 |  40 |
| 7 |    350 |  35 |
| 8 |    500 |  50 |
+-----+-----+-----+

Agent 3 - Optimization Results:
(1, 100, 10, 10.0)
(2, 200, 20, 10.0)
(3, 150, 15, 10.0)
(4, 300, 30, 10.0)
(5, 250, 25, 10.0)
(6, 400, 40, 10.0)
(7, 350, 35, 10.0)
(8, 500, 50, 10.0)

Best Solution:
Item: 1
Resource: 100
Cost: 10
Optimization Score: 10.0

```

3)ANS:

Big Data AI Pipeline Design

Objective

The aim is to design a distributed AI pipeline using Apache Spark RDDs for large-scale data preprocessing, feature engineering, and model training. The pipeline should provide parallel processing, scalability, and fault tolerance.

Architecture

Data Sources

(CSV / IoT / Sensors / Logs)



Spark Cluster



Input RDD



Data Preprocessing

map() / filter()



Feature Engineering

map() / flatMap() / mapValues()



Training Data RDD



Distributed ML Model



Prediction / Evaluation

Data Preprocessing

RDD transformations can clean and prepare large datasets.

Python

Run

```
data = sc.textFile("customer_data.csv")
```

```
clean_data = data.map(lambda x: x.split(",")) \
    .filter(lambda x: len(x) == 5)
```

- map() transforms each record.
- filter() removes invalid records.
- distinct() can remove duplicate records.
- flatMap() can split one record into multiple records.

Feature Engineering

Suppose the dataset contains:

age, income, usage, complaints, churn

Features can be extracted as:

Python

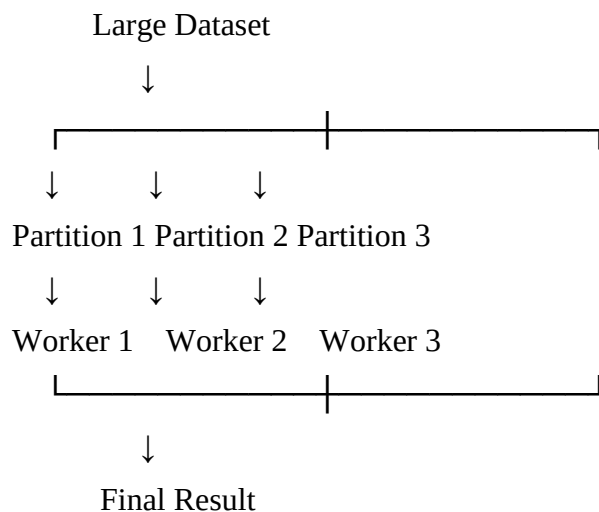
Run

```
features = clean_data.map(  
    lambda x: (  
        float(x[0]),  
        float(x[1]),  
        float(x[2]),  
        float(x[3]),  
        int(x[4])  
    )  
)
```

The resulting RDD contains the numerical features required for machine learning.

Scalability

RDDs divide large datasets into partitions:



More worker nodes can be added when the dataset becomes larger.

7. Fault Tolerance

RDDs are fault tolerant because Spark maintains **RDD lineage**.

If a partition is lost, Spark can recompute it from the original data and transformations:

Input RDD

↓

map()

↓

filter()



Feature RDD



Lost Partition

Thus, failure of one worker does not require the entire pipeline to restart.

8. Advantages

- Handles massive datasets.
- Provides parallel processing.
- Supports distributed machine learning.
- RDD lineage provides fault tolerance.

Conclusion

A distributed AI pipeline based on Spark RDDs provides an efficient approach for preprocessing, feature engineering, and model training. RDD partitioning enables scalability, while lineage provides fault tolerance.

4)ANS:

Real-Time Decision-Making System

1. Objective

The objective is to construct an AI-based real-time decision-making system that combines RDD-based streaming processing with intelligent agents.

A Smart Traffic Management System is used as an example.

The system receives real-time information such as:

- Vehicle count
- Average speed
- Traffic density
- Waiting time

System Architecture

Traffic Sensors / Cameras



Real-Time Data



Spark Streaming



RDD Batches



Data Preprocessing



Feature Extraction

↓

↓

Coordination Agent

↓

Intelligent Decision

↓

Traffic Signal / Alert

RDD-Based Streaming

Spark Streaming divides incoming data into small batches. Each batch can be processed as an RDD.

Python

Run

```
from pyspark.streaming import StreamingContext
```

```
ssc = StreamingContext(sc, 5)
```

```
stream = ssc.socketTextStream("localhost", 9999)
```

```
traffic_rdd = stream.map(lambda x: x.split(","))
```

```
traffic_rdd = traffic_rdd.filter(  
    lambda x: len(x) >= 4  
)
```

Here, traffic data is processed every 5 seconds.

Intelligent Agents

Traffic Monitoring Agent

Collects and analyzes:

Vehicle Count

+

Average Speed

+

Waiting Time

↓

Traffic Condition

Signal Control Agent

Determines signal timing based on traffic density.

Density < 30 → Green 30 seconds

Density 30–60 → Green 60 seconds

Density > 60 → Green 90 seconds

Accident Agent

If an accident is detected:

Accident Detected



Send Emergency Alert



Give Emergency Vehicle Priority



Change Traffic Signals

Coordination Agent

The coordination agent receives information from all agents and selects the final action.

Example

Suppose the system receives:

Road

	Vehicles	Speed	Decision
--	----------	-------	----------

Road1	25	45 km/h	Green 30 sec
-------	----	---------	--------------

Road 2	80	15 km/h	Green 90 sec
--------	----	---------	--------------

Road 3	50	30 km/h	Green 60 sec
--------	----	---------	--------------

Road 4	15	50 km/h	Green 30 sec
--------	----	---------	--------------

Road 2 has the highest traffic density, so the Signal Agent gives it the longest green time.

Fault Tolerance

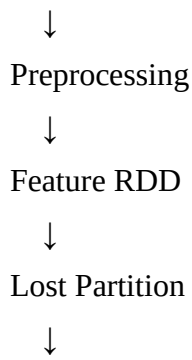
RDD lineage allows Spark to recompute lost partitions.

For example:

Sensor Data



RDD



Recompute Partition

If an individual worker fails, Spark can recover the lost data partition from its lineage.

Scalability

The system can support thousands of sensors by:

- Increasing Spark worker nodes.
- Increasing RDD partitions.
- Running agents in parallel.

Benefits

- Real-time traffic monitoring.
- Fast distributed processing.
- Intelligent autonomous decisions.
- Automatic signal control.

5)ANS:

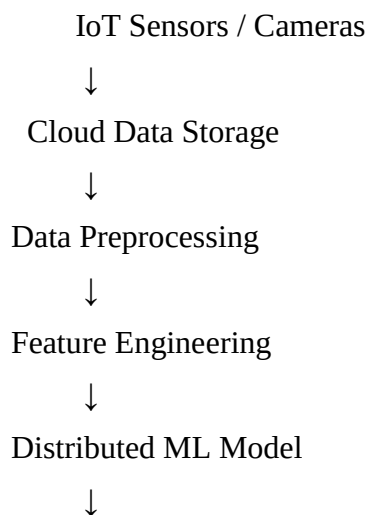
Cloud-Based Distributed AI Application

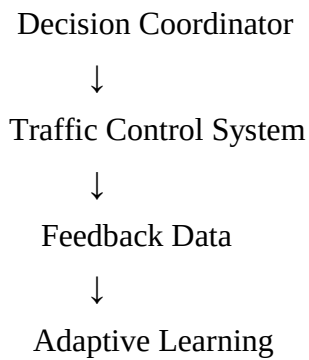
Objective

The objective is to design a cloud-enabled distributed AI system using PySpark and intelligent agents to process large-scale data, dynamically allocate cloud resources, and continuously improve its learning model.

A suitable application is a Cloud-Based Smart Traffic Management System.

System Architecture





Main Components

A. Data Sources

The system collects large amounts of data from:

- Traffic cameras
- IoT sensors
- GPS devices

Example data:

Road_ID, Vehicle_Count, Speed, Waiting_Time, Accident

R1, 80, 20, 45, 0

R2, 35, 45, 10, 0

R3, 100, 15, 60, 1

Distributed Machine Learning

A machine-learning model can be trained using Spark MLlib.

For example, a traffic prediction model can predict whether traffic is Low, Medium, or High.

Program:

```
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.classification import RandomForestClassifier
assembler = VectorAssembler(
    inputCols=[
        "Vehicle_Count",
        "Speed",
        "Waiting_Time"
    ],
    outputCol="features"
)
data = assembler.transform(df)
train, test = data.randomSplit([0.8, 0.2], seed=42)
```

```

model = RandomForestClassifier(
    featuresCol="features",
    labelCol="Accident"
)
trained_model = model.fit(train)
predictions = trained_model.transform(test)
predictions.select(
    "features", "prediction"
).show()

```

Decision Agent

Combines information from other agents and selects the appropriate action.

Traffic Agent

+

Prediction Agent

+

Resource Agent

↓

Decision Agent

↓

Final Action

Learning Agent

Monitors the results of previous decisions and updates the learning process.

Prediction

↓

Actual Result

↓

Compare

↓

Model Improvement

Adaptive Learning

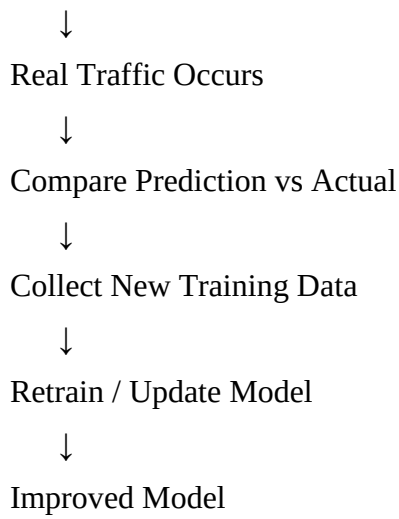
The system continuously receives new data.

For example:

Initial Model

↓

Predict Traffic



Advantages

Scalability

PySpark can distribute large datasets across many cloud worker nodes.

Dynamic Resource Allocation

Cloud resources can increase or decrease according to workload.

Adaptive Learning

The AI model can learn from newly collected data.

Conclusion

A Cloud-Based Distributed AI Application combines cloud computing, PySpark, machine learning, and intelligent agents into a scalable AI architecture. PySpark performs large-scale distributed data processing and model training, while intelligent agents monitor traffic, predict future conditions, manage resources, and coordinate decisions.